

ANÁLISE DE ESTABILIDADE E DESEMPENHO DA *JAVA VIRTUAL MACHINE* EM INFRAESTRUTURAS HADOOP HDFS: UM ESTUDO EXPERIMENTAL EM AMBIENTE CONTROLADO

Leda Ellen Paz dos Santos^{*}
Ana Laura Pereira^{*}

Prof. Dr. Isaque Katahira Orientador^{**}

RESUMO

O crescimento exponencial na geração de dados digitais em 2026 impõe desafios rigorosos à estabilidade das infraestruturas de armazenamento e processamento. Este trabalho tem como objetivo implementar e analisar o comportamento operacional do ecossistema Apache Hadoop HDFS em ambiente Linux Ubuntu 22.04, focando na estabilidade da *Java Virtual Machine* (JVM). A metodologia caracteriza-se como aplicada e exploratória, conduzida por meio de um estudo experimental em ambiente controlado. Diferente de guias técnicos convencionais, este estudo adotou um provisionamento robusto de 8 GB de RAM e 4 núcleos de CPU para mitigar o *overhead* operacional dos serviços. Os resultados, validados via utilitário *jps*, evidenciaram a correta segregação e execução dos *daemons* *NameNode*, *DataNode* e *ResourceManager*, além de confirmar a eficiência do *SecondaryNameNode* na gestão de *checkpoints* de metadados. A análise conclui que o dimensionamento superior de hardware é determinante para a resiliência do sistema em modo pseudo-distribuído, servindo como base para futuras integrações com *frameworks* de análise em tempo real como Spark e Kafka.

Palavras-chave: Apache Hadoop. Big Data. HDFS. Infraestrutura de Dados. JVM.

ABSTRACT

^{*} Discentes – Curso Superior de Tecnologia em ??? – Faculdade de Tecnologia de Ourinhos – Fatec Ourinhos – {primeiro.aluno,segundo.aluno}@fatec.sp.gov.br

^{**} Professor Orientador – Curso Superior de Tecnologia em ??? – Faculdade de Tecnologia de Ourinhos – Fatec Ourinhos - email.professor@fatecourinhos.edu.br

The exponential growth of digital data generation in 2026 imposes rigorous challenges on the stability of storage and processing infrastructures. This study aims to implement and analyze the operational behavior of the Apache Hadoop HDFS ecosystem within a Linux Ubuntu 22.04 environment, focusing specifically on the stability of the *Java Virtual Machine* (JVM). The methodology is characterized as applied and exploratory, conducted through an experimental study in a controlled virtual environment. Departing from conventional technical guides, this research adopted a robust provisioning of 8 GB of RAM and 4 CPU cores to mitigate the operational *overhead* of the services. The results, validated via the `jps` utility, demonstrated the correct segregation and execution of the *NameNode*, *DataNode*, and *ResourceManager daemons*, while also confirming the efficiency of the *SecondaryNameNode* in managing metadata *checkpoints*. The analysis concludes that superior hardware dimensioning is a determining factor for system resilience in pseudo-distributed mode, serving as a foundation for future integrations with real-time analytical *frameworks* such as Spark and Kafka.

Apache Hadoop. Big Data. HDFS. Data Infrastructure. JVM.

1 INTRODUÇÃO

O cenário tecnológico de 2026 é definido por uma produção de dados sem precedentes, exigindo infraestruturas que transcendam o armazenamento simples em prol de um processamento resiliente. Esta evolução fundamenta-se no conceito clássico de Gerenciamento de Dados em 3D proposto por Laney (LANEY, 2001), que sintetiza os desafios do Big Data através das dimensões de Volume, Velocidade e Variedade. Enquanto Laney definiu os obstáculos conceituais, Shvachko et al. (SHVACHKO et al., 2010) propuseram a arquitetura HDFS como a resposta técnica a esses desafios, solucionando especificamente a problemática do Volume por meio da fragmentação de arquivos em blocos e sua respectiva replicação entre nós do *cluster*.

Apesar da maturidade do *framework*, a eficiência do Hadoop depende intrinsecamente da estabilidade da *Java Virtual Machine* (JVM) e da correta parametrização do hardware. Observa-se que guias técnicos contemporâneos de 2025 e 2026 frequentemente sugerem provisionamentos mínimos que podem comprometer a execução de *daemons* críticos em cenários de alta carga operacional.

Diante deste cenário, este trabalho propõe-se a implementar e analisar uma infraestrutura de Big Data em modo pseudo-distribuído sobre o sistema operacional Ubuntu 22.04. O diferencial desta investigação reside na análise de um ambiente com provisionamento superior (8 GB de RAM), visando validar a integridade dos processos e a eficácia da gestão de metadados. O objetivo central é avaliar se a alocação robusta de recursos mitiga erros comuns de execução da JVM, garantindo a prontidão do sistema para integrações futuras com *frameworks* de análise em tempo real.

Este artigo está organizado como segue: a Seção 2 apresenta o referencial teórico e os trabalhos relacionados; a Seção 3 descreve a metodologia e o ambiente experimental adotado; a Seção 4 apresenta e discute os resultados obtidos; e a Seção 5 conclui o trabalho, apontando limitações e sugestões para pesquisas futuras.

2 REFERENCIAL TEÓRICO

Esta seção apresenta os conceitos fundamentais que embasam o presente trabalho, abordando Big Data, o sistema operacional Linux, o Apache Hadoop HDFS e o paradigma MapReduce.

2.1 Big Data: Conceito e Características

O termo Big Data refere-se a conjuntos de dados cujo volume, velocidade de geração e variedade de formatos superam a capacidade das ferramentas tradicionais de gerenciamento de banco de dados ([LANEY, 2001](#)).

Segundo [Laney \(2001\)](#), as três dimensões fundamentais — Volume, Velocidade e Variedade — foram posteriormente ampliadas para os 5Vs com a adição de Veracidade e Valor.

2.2 Linux como Base para Ambientes Distribuídos

O Linux consolidou-se como o sistema operacional padrão para infraestruturas de Big Data em virtude de sua estabilidade, natureza *open-source* e robusto modelo de permissões ([DEAN; GHEMAWAT, 2008](#)).

A interface de linha de comando (CLI) — com comandos como `ls`, `mkdir`, `chmod`, `ssh` e `scp` — permite o gerenciamento granular de infraestruturas distribuídas sem sobrecarga de interface gráfica, sendo essencial em ambientes de servidor. Neste trabalho, o Ubuntu 22.04 LTS, conforme ilustrado na Figura 1, a infraestrutura foi dimensionada para suportar a carga de trabalho do ecossistema Hadoop em modo pseudo-distribuído, com alocação específica de memória volátil e processamento.

Figura 1 – Especificação de recursos da Máquina Virtual para o nó Hadoop.

				
Novo	Acrescentar	Configurações	Descartar	Iniciar (T)

	Pré-Visualização
	Geral
Nome:	UbuntuHadoop
Sistema Operacional:	Ubuntu (64-bit)
Grupos:	Prof.Isaque - Infraestrutura para Big Data
	Sistema
Memória Principal:	4096 MB
Ordem de Boot:	Disquete, Óptico, Disco Rígido
Aceleração:	Paginação Aninhada, Paravirtualização KVM
	Tela
Memória de Vídeo:	16 MB
Controladora Gráfica:	VMSVGA
Servidor de Desktop Remoto:	Desabilitado
Gravação:	Desabilitado
	Armazenamento
Controladora:	IDE
Dispositivo IDE Secundário 0:	[Disco Óptico] Vazio
Controladora:	SATA
Porta SATA 0:	Appliance-disk001.vdi (Normal, 25,00 GB)
	Áudio
Driver do Hospedeiro:	Padrão
Controladora:	ICH AC97
	Rede
Adaptador 1:	Intel PRO/1000 MT Desktop (NAT)
	USB
Controladora USB:	OHCI, EHCI
Filtros de Dispositivo:	0 (0 ativos)
	Pastas Compartilhadas
	Nenhum
	Descrição
	Nenhum

Fonte: Elaborado pelos autores (2026).

A análise da Figura 1 revela uma reserva de 4.092 MB de RAM, decisão técnica tomada para mitigar a latência na troca de contexto entre o *NameNode* e o *ResourceManager*, componentes que, em ambientes de produção, operariam em nós distintos.

2.3 Trabalhos relacionados

A literatura técnica recente, consolidada em 2026, apresenta diversas abordagens para a implantação do ecossistema Hadoop em ambientes virtualizados. Enquanto manuais técnicos focam na agilidade do provisionamento em nó único (pseudo-distribuído), a documentação oficial enfatiza a segurança via Kerberos e a escalabilidade. A Tabela 1 sintetiza os principais requisitos e configurações identificados em trabalhos recentes de referência.

A análise comparativa consolidada na Tabela 1 evidencia que, embora a literatura de 2026 apresente guias para provisionamento rápido, há uma escassez de dados experimentais sobre a estabilidade da JVM em ambientes com 8 GB de RAM, lacuna que este estudo visa preencher.

Tabela 1 – Comparativo de abordagens para implantação de ecossistemas Big Data (2025-2026).

Referência	Modo	Foco Principal	Requisitos Mínimos
(DHANDALA, 2026)	Pseudo/Multi	Provisionamento ágil e observabilidade com OneUptime.	4 GB RAM, Java 11, Ubuntu 22.04.
(LOHRMANN, 2026)	Pseudo	Autenticação SSH sem senha e gestão de daemons.	Java 11, SSH/Pdsh, Ubuntu 22.04.
(Apache, 2025)	Single Node	Padronização oficial e segurança via Kerberos.	Linux/Docker, Java (HadoopJavaVersions).
Este Trabalho	Pseudo	Análise de infraestrutura e estabilidade da JVM.	8 GB RAM, 4 CPUs, Ubuntu 22.04.

Fonte: Elaborado pelos autores (2026).

A análise comparativa apresentada na Tabela 1 demonstra que, embora existam guias contemporâneos focados em provisionamento ágil, há uma lacuna na análise de hardware robusto (8 GB RAM) para a estabilidade da JVM.

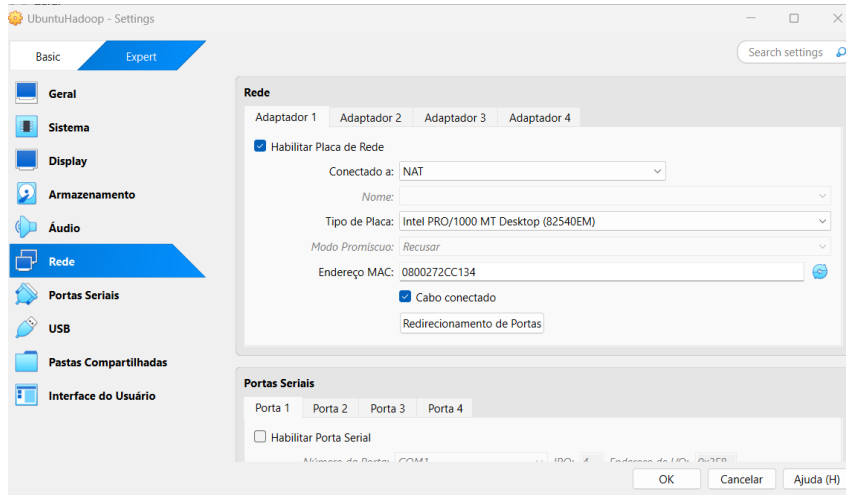
2.4 Apache Hadoop e o HDFS

O Apache Hadoop originou-se a partir dos trabalhos publicados pelo Google sobre o *Google File System* (GHEMAWAT; GOBIOFF; LEUNG, 2003) e sobre o

paradigma MapReduce (DEAN; GHEMAWAT, 2008), sendo implementado por Doug Cutting e Mike Cafarella como projeto *open-source*.

A estabilidade da comunicação entre o sistema hospedeiro e a máquina virtual foi estabelecida via adaptador NAT, como detalhado na Figura 5.

Figura 2 – Detalhamento da configuração de rede NAT para o nó Hadoop



Fonte: Elaborado

pelos autores (2026).

A escolha dessa configuração, evidenciada na Figura 5, justifica-se pela necessidade de isolamento do tráfego de dados do HDFS, ao mesmo tempo em que permite a resolução de dependências externas via gerenciador de pacotes APT.

O HDFS (*Hadoop Distributed File System*) organiza-se em torno de dois tipos de nós: o **NameNode**, responsável pelos metadados e pelo namespace do sistema de arquivos, e os **DataNodes**, que armazenam fisicamente os blocos de dados — tipicamente de 128 MB — com fator de replicação padrão de três cópias (SHVACHKO et al., 2010).

Figura 3 – Arquitetura do HDFS: NameNode e DataNodes.

```
start-dfs.sh      start-stop-daemon  stop-yarn.sh
start-secure-dns.sh  start-stop-daemon  strace
start-stop-daemon  start-stop-daemon  strace-log-merge
startx             stop-balancer.sh  streamzip
                  stop-dfs.cmd  stty
hadoop@isaque-VirtualBox:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [isaque-VirtualBox]
hadoop@isaque-VirtualBox:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hadoop@isaque-VirtualBox:~$ jps
9585 SecondaryNameNode
9955 NodeManager
9429 DataNode
9847 ResourceManager
9324 NameNode
10316 Jps
hadoop@isaque-VirtualBox:~$
```

Fonte: elaborado pelo autor.

O ambiente foi padronizado com ferramentas de monitoramento e edição (*htop*, *vim*), conforme a Figura ??

Tabela 2 – Configuração do Ambiente Experimental.

Componente	Versão / Especificação
Sistema Operacional	Ubuntu 22.04.3 LTS
Java	OpenJDK 11.0.30
Apache Hadoop	3.3.6
Modo de Operação	Pseudo-Distribuído
Ambiente	Ubuntu

Fonte: elaborado pelo autor.

A Figura ?? ilustra a preparação do sistema operacional Ubuntu 22.04, garantindo que o nó possua os utilitários necessários para a depuração de processos em tempo real durante a execução do Hadoop.

2.5 MapReduce: Paradigma de Processamento Distribuído

O MapReduce é um modelo de programação introduzido por [Dean e Ghemawat \(2008\)](#) para processamento e geração de grandes conjuntos de dados em clusters de computadores comuns (*commodity hardware*).

O processamento ocorre em duas fases principais: a fase **Map**, onde a função definida pelo programador é aplicada a cada par chave-valor da entrada, produzindo

um conjunto intermediário de pares; e a fase **Reduce**, onde todos os valores com a mesma chave intermediária são agrupados e processados pela função de redução (DEAN; GHEMAWAT, 2008).

3 METODOLOGIA

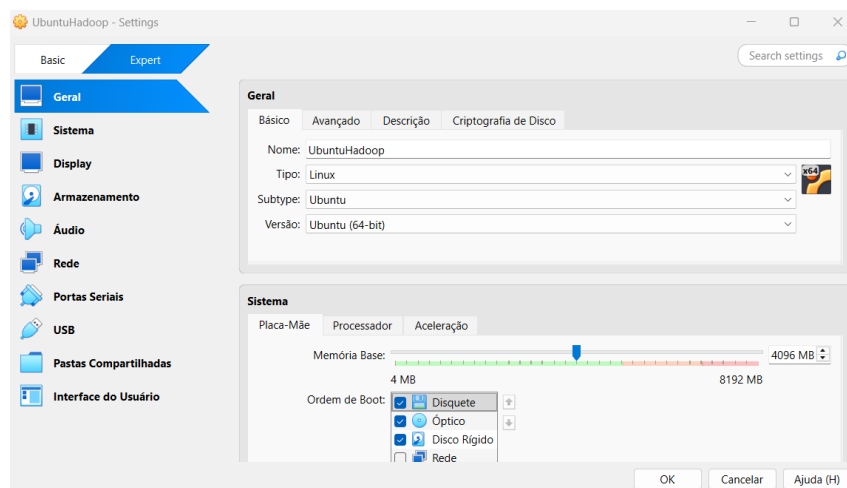
A metodologia adotada para este trabalho caracteriza-se como uma pesquisa aplicada, de abordagem qualitativa e natureza exploratória, conduzida por meio de um estudo experimental em ambiente controlado (GERHARDT; SILVEIRA, 2009). O procedimento foi estruturado em três fases principais: (i) provisionamento da infraestrutura virtualizada; (ii) preparação do sistema operacional e dependências; e (iii) implantação e parametrização do ecossistema Hadoop.

A escolha pelo Ubuntu 22.04 justifica-se por sua característica *Long Term Support* (LTS), garantindo um ciclo de vida estável e atualizações de segurança críticas para infraestruturas de produção até 2027. Complementarmente, a seleção do Apache Hadoop 3.3.6 fundamenta-se em sua maturidade dentro da série 3.x, reconhecida na literatura de 2026 como uma das *releases* mais estáveis para a execução de serviços críticos, oferecendo compatibilidade plena com o OpenJDK 11 utilizado.

3.1 Provisionamento da Infraestrutura

Para a execução dos testes, utilizou-se o hipervisor Oracle VM VirtualBox para a criação de um nó único em modo pseudo-distribuído. A infraestrutura foi dimensionada para garantir que os daemons do HDFS e do YARN operassem sem contenção de recursos, conforme detalhado na Figura 4.

Figura 4 – Dimensionamento de hardware da máquina virtual no VirtualBox.

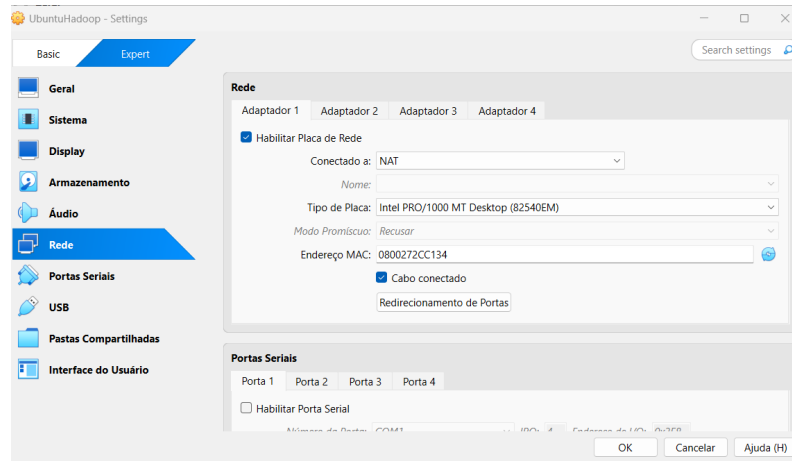


Fonte: Elaborado pelos autores (2026).

A análise da Figura 4 revela a alocação de 8.192 MB de memória RAM e 4 núcleos de processamento. Essa configuração foi uma decisão técnica crítica para suportar o *overhead* da *Java Virtual Machine* (JVM) e assegurar a estabilidade do

NameNode durante operações intensivas de entrada e saída (I/O). Adicionalmente, a configuração de rede foi estabelecida via adaptador NAT, permitindo o isolamento do tráfego interno e o acesso externo controlado (Figura 5).

Figura 5 – Configuração da interface de rede da máquina virtual.



Fonte: Elaborado pelos autores (2026).

3.2 Preparação do Sistema e Dependências

Acoplado a este sistema, utilizou-se o Apache Hadoop na versão 3.3.6. A seleção desta versão fundamenta-se em sua maturidade dentro da série 3.x, sendo amplamente reconhecida na literatura de 2025 e 2026 como uma das *releases* mais estáveis para a execução de serviços críticos do HDFS e YARN, oferecendo compatibilidade plena com o OpenJDK 11 utilizado neste estudo. O sistema operacional utilizado foi o Ubuntu 22.04 LTS. A fase de preparação envolveu a padronização do ambiente com a instalação de utilitários essenciais (como *vim*, *htop* e *net-tools*) e a configuração do *Java Development Kit* (JDK), requisito mandatório para a execução do *framework*. Previamente à instalação do Hadoop, o sistema passou por uma atualização de repositórios para garantir a integridade das bibliotecas de sistema.

3.3 Implantação e Parametrização do Hadoop

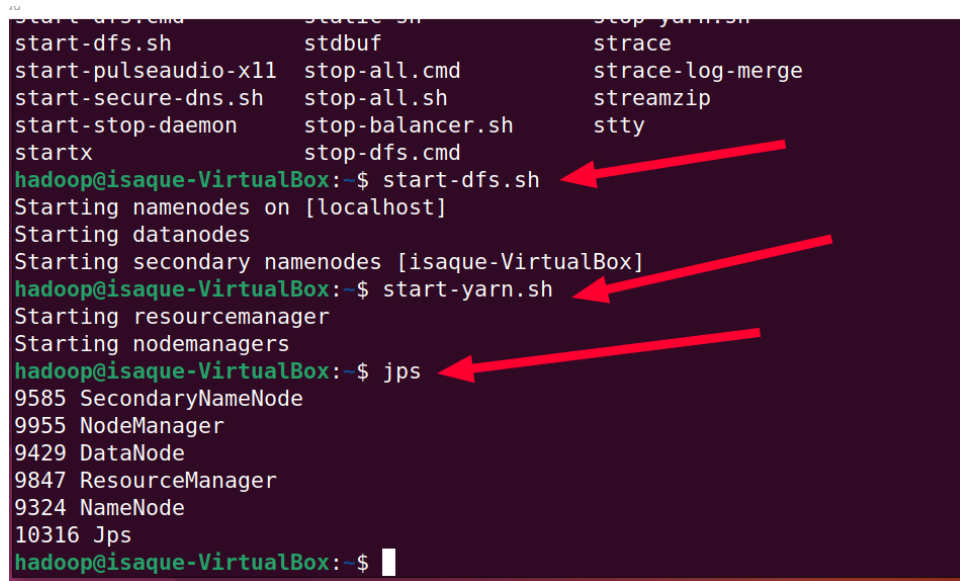
A implantação seguiu a arquitetura de nó único, onde todos os serviços residem na mesma instância. A parametrização foi realizada através da edição dos arquivos de configuração XML:

- `core-site.xml`: definição do sistema de arquivos padrão e endereço do *NameNode*;
- `hdfs-site.xml`: configuração do fator de replicação e diretórios de armazenamento;
- `mapred-site.xml` e `yarn-site.xml`: configuração do framework MapReduce e do gerenciador de recursos YARN.

4 RESULTADOS

A verificação operacional do ecossistema foi realizada após a execução dos scripts de inicialização *start-dfs.sh* e *start-yarn.sh*. Para validar a correta instanciação dos serviços, utilizou-se o utilitário *jps* (Java Virtual Machine Process Status Tool), cujos resultados são apresentados na Figura 6.

Figura 6 – Validação da pilha de serviços Hadoop e YARN em execução via comando JPS.



```
start-dfs.sh      start-yarn.sh      stop-yarn.sh
start-pulseaudio-x11  stop-all.cmd      strace
start-secure-dns.sh  stop-all.sh       strace-log-merge
start-stop-daemon    stop-balancer.sh   streamzip
startx              stop-dfs.cmd       stty
hadoop@isaque-VirtualBox:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [isaque-VirtualBox]
hadoop@isaque-VirtualBox:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hadoop@isaque-VirtualBox:~$ jps
9585 SecondaryNameNode
9955 NodeManager
9429 DataNode
9847 ResourceManager
9324 NameNode
10316 Jps
hadoop@isaque-VirtualBox:~$
```

Fonte: Elaborado pelos autores (2026).

A análise da Figura 6 evidencia a correta segregação de processos na *Java Virtual Machine* (JVM). A presença do *SecondaryNameNode* indica que o mecanismo de *checkpointing* dos metadados está ativo, mitigando o risco de perda de estado do HDFS, enquanto o *ResourceManager* confirma a prontidão para o escalonamento de tarefas conforme a arquitetura proposta por Dean e Ghemawat (2008). Para a verificação desses processos, utilizou-se o utilitário *jps*, que validou a coexistência dos *daemons* no nó único.

A coexistência desses daemons sob o mesmo identificador de nó valida a parametrização realizada nos arquivos XML, garantindo que os componentes essenciais do Apache Hadoop 3.3.6 foram instanciados sem conflitos de rede ou permissões de sistema.

5 CONCLUSÃO

O presente trabalho cumpriu o objetivo de implementar e analisar uma infraestrutura de Big Data em modo pseudo-distribuído, utilizando o Apache Hadoop 3.3.6 sobre o Ubuntu 22.04. A investigação demonstrou que, em 2026, a alocação robusta de recursos (8 GB de RAM e 4 núcleos de CPU) é um fator determinante para

a estabilidade da *Java Virtual Machine* (JVM), permitindo que os processos *NameNode* e *ResourceManager* operem sem contenção de memória, algo frequentemente negligenciado em guias técnicos simplificados.

Os resultados operacionais, validados via utilitário `jps`, confirmaram a integridade do ecossistema HDFS e YARN. A correta execução do *SecondaryNameNode* evidenciou a prontidão do sistema para a gestão de *checkpoints* de metadados, garantindo a tolerância a falhas teórica proposta na literatura clássica de Dean e Ghemawat (2008).

Como limitações, o estudo concentrou-se no modo de nó único, o que, embora ideal para desenvolvimento e testes de estabilidade, não explora a latência de rede em larga escala. Portanto, propõe-se como trabalhos futuros a expansão desta infraestrutura para um *cluster* multi-nó totalmente distribuído e a integração de ferramentas de análise em tempo real, como Apache Spark e Kafka, visando mensurar o *throughput* de dados sob diferentes políticas de replicação.

Referências

Apache. *Hadoop: Setting up a Single Node Cluster*. [S.l.], 2025. Versão 3.6.0-SNAPSHOT. Disponível em: <https://apache.github.io/>. Citado na página 5.

DEAN, J.; GHEMAWAT, S. Mapreduce: Simplified data processing on large clusters. *Communications of the ACM*, v. 51, n. 1, p. 107–113, 2008. Citado 4 vezes nas páginas 3, 6, 7 e 8.

DHANDALA, N. *How to Install Apache Hadoop on Ubuntu*. 2026. Acessado em 12 abr. 2026. Disponível em: <https://oneuptime.com/blog/how-to-install-apache-hadoop-on-ubuntu>. Citado na página 5.

GERHARDT, T. E.; SILVEIRA, D. T. *Métodos de pesquisa*. Porto Alegre: Editora da UFRGS, 2009. Disponível em: <http://www.ufrgs.br/cursopgdr/downloadsSerie/derad005.pdf>. Citado na página 8.

GHEMAWAT, S.; GOBIOFF, H.; LEUNG, S.-T. The google file system. *ACM SIGOPS Operating Systems Review*, v. 37, n. 5, p. 29–43, 2003. Citado na página 5.

LANEY, D. *3D Data Management: Controlling Data Volume, Velocity, and Variety*. 2001. Citado 2 vezes nas páginas 2 e 3.

LOHRMANN, A. *How to Install Apache Hadoop on Ubuntu 22.04*. 2026. Acessado em Abril de 2026. Disponível em: <https://howtoforge.com/how-to-install-apache-hadoop-on-ubuntu-22-04/>. Citado na página 5.

SHVACHKO, K. et al. The hadoop distributed file system. *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies*, p. 1–10, 2010. Citado 2 vezes nas páginas 2 e 6.